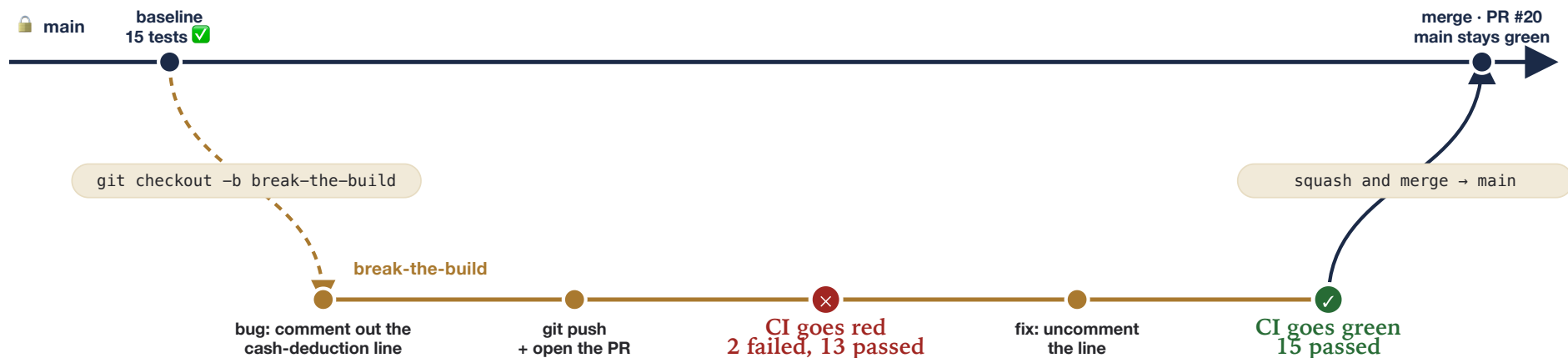


From *main* to *break-the-build*, and back

Here's how the code moves in the CI demo: a short-lived branch is created off *main*, picks up commits, the GitHub Actions check stops it in red, it gets fixed, and only then does it go back into *main* through a merge.

main is never touched directly at any point in this loop — that rule is exactly why the diagram has this shape.



The **main** line never breaks or changes color: it runs straight from left to right. The only thing that ever happens to it is one new commit at the end — the merge — and that only lands because the commit right before it on **break-the-build** was green.

— main — protected, never takes a direct push

— break-the-build — working branch, deleted after the merge

● check is red → Merge button disabled

● check is green → Merge button enabled

Maps to sections 2–6 of **DEMO_SCRIPT.md**: branch and break the build, open the PR, watch the check go red, fix it and watch it go green, then merge.

Mermaid source, if you want to paste it into a .md

Same story, as gitGraph — drop it straight into a ``mermaid`` fence in WALKTHROUGH.md or DEMO_SCRIPT.md.

COPY THIS BLOCK

```
%%{init: {'theme':'base','themeVariables':{'git0':'#1b2a47','git1':'#a9792f','commitLabelColor':'#1b2a47','commitLabelBackground':'#faf5e9'}}}%%
gitGraph
    commit id: "baseline: 15 passed"
    branch break-the-build
    commit id: "bug: comment out cash deduction"
    commit id: "push + open PR"
    commit id: "CI red: 2 failed" type: HIGHLIGHT
    commit id: "fix: uncomment the line"
    commit id: "CI green: 15 passed" type: HIGHLIGHT
    checkout main
    merge break-the-build id: "squash and merge (PR #20)"
    commit id: "main protected, back to green"
```